

SOFTWARE REQUIREMENTS SPECIFICATION

BTCUSDT Daily OHLCV Analysis & Next-Day Price Prediction System

SRS for Kaggle Notebook Implementation

Document Item	Specification
Dataset	BTCUSDT Daily OHLCV Data
Kaggle Source	https://www.kaggle.com/datasets/trevorfrench/btcusdt-daily-ohlcw-data
Implementation Platform	Kaggle Notebook (.ipynb)
Dataset Snapshot	1,909 daily records; 2017-10-25 to 2023-01-15
Dataset Fields	id, open, close, low, high, amount, vol, count
Primary Task	Next-day BTCUSDT price / return prediction
Secondary Task	Market-direction classification and exploratory analysis
Document Purpose	Define a practical, reproducible ML project before notebook development

Important scope note: This is an analytical/predictive ML project, not a financial-advice or automated-trading system. Predictions are experimental and must not be interpreted as guaranteed future prices or profits.

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements, scope, data processing, machine-learning workflow, evaluation strategy, notebook structure, and expected outputs for a BTCUSDT daily market prediction project. The SRS is intentionally designed around the supplied Kaggle dataset so that it can be implemented as a single Kaggle Notebook (.ipynb).

1.2 Project Objective

The system shall analyze historical Bitcoin/USDT daily OHLCV-style market data and build machine-learning models that use information available up to day t to estimate the market state on day $t+1$. The notebook shall compare baseline and ML models, avoid temporal data leakage, report appropriate regression and classification metrics, and identify the most useful market features.

1.3 Intended Users

- AI/ML students implementing a complete end-to-end forecasting project.
- Researchers or learners studying time-series feature engineering and model evaluation.
- Kaggle users interested in reproducible BTCUSDT exploratory analysis and prediction.
- The project is not intended to execute real-money trades or provide financial advice.

2. Dataset Specification

The supplied CSV contains 1,909 records and 8 columns. The timestamp field is stored as a Unix timestamp. The observed date range in the supplied file is 2017-10-25 through 2023-01-15. The current file contains 0 missing values and no duplicate rows.

Field	Type	Meaning	Use
id	Integer	Unix timestamp for the daily record	Convert to datetime; primary time index
open	Float	Opening BTCUSDT price for the day	Feature / historical price
high	Float	Highest BTCUSDT price for the day	Feature / volatility information
low	Float	Lowest BTCUSDT price for the day	Feature / volatility information
close	Float	Closing BTCUSDT price for the day	Target source / feature
amount	Float	Traded quantity/amount represented by the source dataset	Volume-related feature
vol	Float	Trading volume/value represented by the source dataset	Volume-related feature
count	Integer	Number of trades/transactions represented by the source dataset	Activity feature

The notebook must retain the original field names unless a clearly documented rename is applied. The exact semantic interpretation of amount/vol/count should be verified against the Kaggle dataset description before final publication.

3. Scope

3.1 In Scope

- Load the Kaggle CSV into pandas.
- Inspect data types, time ordering, missing values, duplicates, and descriptive statistics.
- Convert the Unix timestamp into a datetime index/column and sort records chronologically.
- Perform EDA using price, return, volatility, and volume/activity visualizations.
- Engineer lag, rolling, return, range, volatility, and volume/activity features using past information only.
- Create a next-day regression target and a next-day direction classification target.

- Build naive/time-series baselines and multiple ML models.
- Use chronological train/validation/test splitting; do not randomly shuffle time-series observations.
- Evaluate models using regression and classification metrics.
- Compare feature importance/model interpretability where supported.
- Produce a final prediction table and concise findings in the notebook.
- Save the trained best model and/or feature list when practical within Kaggle constraints.

3.2 Out of Scope

- Live exchange API integration.
- Real-time streaming prediction.
- Automatic order execution or portfolio management.
- Guaranteed profit, investment advice, or financial recommendations.
- High-frequency/intraday prediction; the supplied project is daily-frequency.
- External news, social-media, macroeconomic, or on-chain data unless explicitly added as a separate extension.

4. Functional Requirements

ID	Requirement	Description
FR-01	Data Loading	The notebook shall load the supplied BTCUSDT CSV and display its shape, columns, and sample records.
FR-02	Data Validation	The notebook shall check missing values, duplicates, invalid numeric values, and chronological ordering.
FR-03	Timestamp Processing	The notebook shall convert id from Unix timestamp to a readable datetime and sort records ascending by date.
FR-04	EDA	The notebook shall visualize OHLC prices, returns, trading activity, and major distribution characteristics.
FR-05	Feature Engineering	The notebook shall create lagged and rolling features using only information available before the prediction day.
FR-06	Target Creation	The primary regression target shall represent the next-day close or next-day return. The final notebook shall explicitly state which target is selected.
FR-07	Direction Target	The notebook shall optionally create a binary target: 1 if next-day close is higher than the current close, otherwise 0.
FR-08	Time-Series Split	The notebook shall split data chronologically into train, validation, and test sets.
FR-09	Baseline	The notebook shall include at least one naive baseline, such as next-day close = current-day close, for meaningful comparison.
FR-10	Model Training	The notebook shall train multiple suitable ML models and compare their performance.
FR-11	Evaluation	The notebook shall calculate regression and/or classification metrics appropriate to the selected targets.
FR-12	Visualization	The notebook shall plot actual vs predicted values, residual/error behavior, and model comparison results.
FR-13	Interpretability	The notebook shall report feature importance using model-native importance or permutation importance where feasible.
FR-14	Final Output	The notebook shall present the selected best model, test performance, key features, limitations, and example next-day predictions.

5. Non-Functional Requirements

ID	Requirement	Description
NFR-01	Reproducibility	Notebook execution should reproduce results using fixed random seeds where supported.
NFR-02	Leakage Prevention	No future target value or future-derived statistic may be used to create a feature for an earlier observation.
NFR-03	Performance	The complete notebook should run within practical Kaggle Notebook CPU/GPU limits; GPU is optional.
NFR-04	Clarity	Each notebook section shall have Markdown explanations and meaningful chart titles/axis labels.
NFR-05	Robustness	The notebook should handle missing/invalid records explicitly rather than silently dropping data.
NFR-06	Traceability	Feature formulas, target definition, split dates, models, and evaluation metrics shall be documented.

NFR-07	Safety	Results shall be presented as experimental forecasts and not as guaranteed financial outcomes.
--------	--------	--

6. Proposed ML Problem Definition

6.1 Primary Regression Task

Recommended primary target: next-day return rather than raw next-day price. Define $\text{return}(t+1) = (\text{Close}(t+1) / \text{Close}(t)) - 1$. This formulation is generally easier to compare across different BTC price regimes. The notebook may additionally convert predicted return into an estimated next-day close for presentation.

6.2 Secondary Classification Task

Define $\text{direction}(t+1) = 1$ when $\text{Close}(t+1) > \text{Close}(t)$, otherwise 0. This supports a directional classification experiment and provides an intuitive comparison of whether the model can identify upward versus non-upward next-day movement.

6.3 Recommended Features

Feature Group	Examples	Purpose
Raw OHLC	open, high, low, close	Current market price structure
Lagged price	close_lag_1, close_lag_2, close_lag_7	Recent temporal dependence
Returns	return_1d, return_3d, return_7d, return_14d	Momentum and movement
Range	high-low, (high-low)/close	Daily price spread / volatility proxy
Rolling statistics	rolling mean/std over 7, 14, 30 days	Trend and volatility regime
Volume/activity	amount, vol, count and their lags/changes	Market participation/activity
Calendar	day_of_week, month	Potential seasonal effects

All rolling statistics must be shifted appropriately when necessary so that the feature for day t does not incorporate information from t+1 or later.

7. Data Preprocessing Requirements

1. Load CSV with pandas.
2. Inspect schema and descriptive statistics.
3. Convert id to datetime and sort ascending.
4. Check duplicates and missing values.
5. Verify OHLC consistency (for example, high should not be below open/close and low should not be above open/close). Flag anomalous records rather than automatically deleting them without justification.
6. Create derived features from historical observations.
7. Drop rows only after target/lag/rolling construction when the missingness is caused by unavailable history.
8. Keep a clean feature matrix X and target y.
9. Fit scaling/imputation transformations only on the training data when a model requires them.

8. Model Requirements

The project should compare simple baselines against several ML approaches rather than relying on one model.

Model	Task	Reason for Inclusion
Naive persistence	Regression	Essential baseline: next-day value related to today's value
Linear Regression / Ridge	Regression	Simple, interpretable benchmark
Random Forest Regressor	Regression	Nonlinear relationships and feature interactions
Gradient Boosting / HistGradientBoosting	Regression	Strong tabular baseline for nonlinear patterns
Random Forest Classifier	Classification	Directional prediction benchmark
Logistic Regression	Classification	Simple probabilistic directional baseline
Gradient Boosting Classifier	Classification	Nonlinear directional model

Optional extension: XGBoost/LightGBM/CatBoost may be evaluated if available in the Kaggle environment, but they should not be mandatory for the core notebook.

9. Evaluation Requirements

9.1 Regression Metrics

Metric	Use
MAE	Average absolute prediction error; easy to interpret
MSE	Penalizes larger errors more strongly
RMSE	Error in the same units as the target; sensitive to large errors
R ²	Explains variance relative to a constant baseline; should not be used alone
Directional Accuracy	Percentage of cases where predicted return direction matches actual direction

9.2 Classification Metrics

Metric	Use
Accuracy	Overall proportion of correct directions
Precision	Correct positive predictions among all positive predictions
Recall	Correct positive predictions among all actual positives
F1-score	Balance of precision and recall
ROC-AUC	Ranking/discrimination quality across classification thresholds
Confusion Matrix	Detailed view of correct and incorrect directional predictions

The final model must be selected using validation performance and then evaluated once on the untouched chronological test set. Avoid choosing a model based only on the final test result.

10. Time-Series Validation Strategy

Random train_test_split with shuffle=True shall not be used for the primary experiment because it can allow future market information to influence model development. A chronological split is required.

Stage	Suggested Portion	Purpose
Training	70%	Fit model and preprocessing parameters
Validation	15%	Compare models/hyperparameters
Test	15%	Final unbiased chronological evaluation

Exact boundaries should be reported as dates in the notebook. For a stronger extension, use walk-forward or TimeSeriesSplit validation on the training/validation portion.

11. Kaggle Notebook Structure

Section	Notebook Cell/Heading	Expected Content
01	Project Title & Objective	Explain problem, target, dataset source, and limitations.
02	Import Libraries & Configuration	pandas, numpy, matplotlib, seaborn, sklearn and optional libraries; random seed.
03	Load Dataset	Read CSV and display shape, columns, head/tail.
04	Data Understanding	Data types, statistics, missing values, duplicates, timestamp range.
05	Data Cleaning	Datetime conversion, sorting, consistency checks, anomaly handling.
06	Exploratory Data Analysis	Price trend, returns, distributions, volatility, volume/activity charts.
07	Feature Engineering	Lagged features, returns, rolling statistics, ranges, activity changes.
08	Target Creation	Next-day return and direction; remove rows without future targets.
09	Chronological Split	Train/validation/test by time; print exact dates and sizes.
10	Preprocessing	Scaling only where required; fit transformations on training data.
11	Baseline Models	Persistence baseline and simple statistical/linear benchmark.
12	ML Model Training	Train candidate regression/classification models.

13	Model Evaluation	Metrics, comparison tables, confusion matrix, actual-vs-predicted plots.
14	Feature Importance	Feature importance/permutation importance and interpretation.
15	Best Model	Select model using validation results and evaluate on test data.
16	Prediction Demo	Generate example next-day prediction from the latest eligible observation.
17	Conclusion & Limitations	Summarize findings, weaknesses, and future improvements.

12. Visualizations

- BTCUSDT closing price over time.
- OHLC line/candlestick-style visualization if practical.
- Daily return time series.
- Return distribution/histogram.
- Rolling volatility (for example, 7-day and 30-day).
- Trading volume/activity over time.
- Correlation heatmap of engineered numeric features.
- Actual vs predicted target on the test period.
- Prediction error/residual plot.
- Model performance comparison chart.
- Confusion matrix and ROC curve for the classification task.
- Top feature-importance chart.

13. Expected Outputs

- Cleaned and chronologically ordered BTCUSDT dataset.
- EDA findings and market-behavior visualizations.
- Engineered feature dataset.
- Chronological train/validation/test split.
- Baseline performance.
- Comparison of candidate regression models.
- Optional comparison of candidate direction-classification models.
- Final metric table containing validation and test results.
- Actual-vs-predicted visualization.
- Feature importance / model interpretation.
- Example next-day prediction using the most recent eligible data.
- Final conclusion describing whether the model adds value over the naive baseline.

14. Acceptance Criteria

- Notebook executes from top to bottom in Kaggle without unresolved errors.
- Dataset is loaded and its schema is explicitly documented.
- Timestamp is correctly converted and records are chronologically ordered.
- Missing values and duplicates are checked and documented.
- At least one leakage-safe historical feature is engineered.
- Primary next-day target is explicitly defined with its mathematical formula.
- Train/validation/test sets are chronological.
- At least one naive baseline and at least two ML models are evaluated.
- Final evaluation uses an untouched chronological test period.
- At least three relevant regression metrics are reported for the regression task.
- If classification is implemented, at least accuracy, precision, recall, F1, and confusion matrix are reported.

- Model comparison is presented in a clear table and at least two useful prediction/evaluation plots are included.
- The notebook states limitations and does not claim guaranteed financial profitability.

15. Risks and Limitations

Risk / Limitation	Mitigation
Non-stationary market	Use chronological testing and report performance by period where useful.
Data leakage	Shift lag/rolling features and use time-based validation.
Overfitting	Use simple baselines, validation data, regularization and restrained hyperparameter search.
Regime changes	Discuss historical-period dependence and avoid claiming universal performance.
Class imbalance	Report precision/recall/F1 and inspect class distribution.
Metric limitations	Use multiple metrics rather than a single score.
Financial uncertainty	Clearly label outputs as experimental predictions, not investment advice.
Limited feature set	Treat external data such as news/on-chain indicators as future work.

16. Future Enhancements

- Walk-forward evaluation and rolling retraining.
- Hyperparameter optimization using time-series cross-validation.
- XGBoost/LightGBM/CatBoost comparison.
- LSTM/GRU/Temporal CNN models as a separate deep-learning experiment.
- Multi-step forecasting (3-day, 7-day horizons).
- Prediction intervals or probabilistic forecasting.
- Additional technical indicators such as RSI, MACD, ATR and moving-average relationships.
- External features such as on-chain metrics, macroeconomic variables and sentiment/news data.
- Backtesting a clearly defined strategy separately from the predictive ML experiment, including transaction costs and slippage.
- Model monitoring and drift detection for future/live deployment.

17. Recommended Final Notebook Conclusion

The final notebook should answer four practical questions: (1) What patterns and characteristics are visible in the historical BTCUSDT data? (2) Can historical OHLCV-derived features predict next-day return or direction better than a simple baseline? (3) Which model and features perform best on an unseen chronological test period? and (4) What limitations prevent the result from being treated as a reliable trading system?

18. References

Dataset: Trevor French, “BTCUSDT Daily OHLCV Data,” Kaggle.

Kaggle URL: <https://www.kaggle.com/datasets/trevorfrench/btcusdt-daily-ohlc-v-data>

Implementation environment: Kaggle Notebooks / Python.

Core libraries: pandas, numpy, matplotlib, seaborn, scikit-learn; optional gradient-boosting libraries where available.